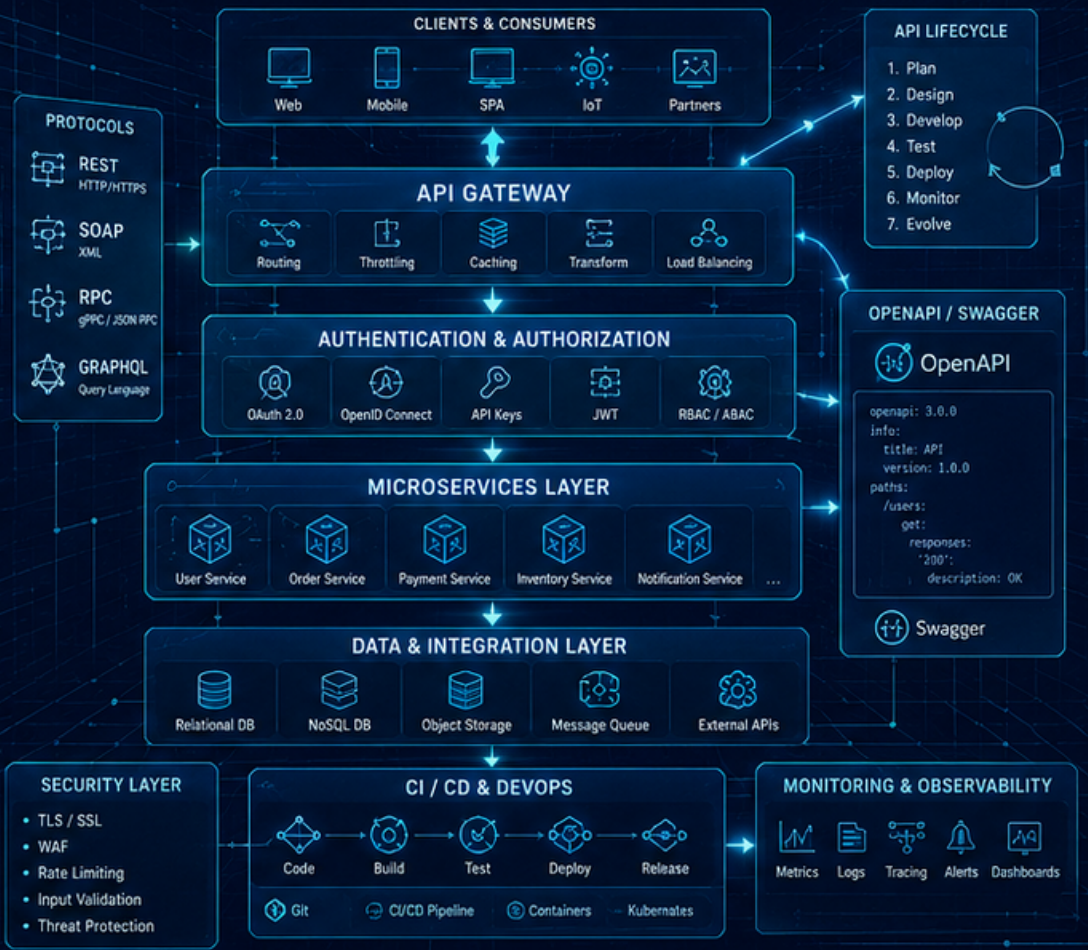
FUNDAMENTALS OF API DEVELOPMENT

ARCHITECT • DESIGN • BUILD • SECURE • DEPLOY • EVOLVE



Kangeyan Passoubady

Kangs | Kavin School

Fundamentals of API Development

Table of Contents

Welcome to Fundamentals of API Development

To ensure a smooth and productive learning experience, please complete the setup before class.

Course Overview

Review the course outline and schedule here: Course Catalog - Fundamentals of API Development

This is a 16-hour course delivered as four 4-hour sessions across four days (morning and afternoon cohorts). You'll build hands-on skills in API design and development using a shared Spring Boot REST API example, with Postman for testing, JUnit5 for automated tests, and IntelliJ or VS Code as the IDE throughout.

Pre-Class Setup (Required)

Please complete the installation before the training session: Installation Instructions

Category	Tools / Packages
JDK	OpenJDK 17+ (recommended: JDK 21)
Build Tool	Apache Maven 3.9+
IDE	IntelliJ IDEA (Community or Ultimate) **or** VS Code with Java & Spring Boot Extension Packs
API Client	Postman
Version Control	Git

The installation guide includes OS-specific instructions for both Windows and macOS.

Once installed, use the quickstart-project to verify everything works end to end. Open it in IntelliJ or VS Code, run it with Maven, call its endpoints from Postman, and run its JUnit5 tests.

Checklist Before Class

- [ ] JDK 17+ installed (`java -version`)
- [ ] Maven 3.9+ installed (`mvn -version`)
- [ ] Git installed (`git --version`)

- [ ] IntelliJ IDEA or VS Code installed and opens `quickstart-project` as a Maven project
- [ ] Postman installed and able to send a request
- [ ] `mvn spring-boot:run` starts `quickstart-project` on http://localhost:8080
- [ ] `GET http://localhost:8080/api/v1/health` returns `{"status":"UP", ...}`
- [ ] `mvn test` passes all JUnit5 tests

Quick Links

Resource	Link
Course Catalog	Fundamentals of API Development
Installation Guide	install.md
macOS Install Guide	install-mac.md
Windows Install Guide	install-win.md
Quickstart Project	quickstart-project

Need Help?

If you encounter any issues during setup, please don't hesitate to reach out before the training day.

Looking forward to seeing you in class.

Fundamentals of API Development - Outline

Duration: 4 Sessions × 4 Hours (16 hours total, split across 4 days, AM and PM cohorts)

Audience: Software developers who need the fundamentals of API design and development

Track: Java (Spring Boot). Hands-on labs are built against a Spring Boot REST API, tested with JUnit5 and Postman, using IntelliJ as the IDE.

Prerequisites: - Basic Java familiarity (classes, interfaces, collections) - IntelliJ, a JDK, and Postman installed before class - No prior API design experience required

Delivery Model: 1 Course, 2 Cohorts, 4 Days

This is authored as a single 16-hour course, but delivered as four 4-hour sessions on four separate days so two cohorts (morning and afternoon) can each go through the full course without doubling instructor prep:

	Day 1	Day 2	Day 3	Day 4
Morning Cohort	Session 1 (AM)	Session 2 (AM)	Session 3 (AM)	Session 4 (AM)
Afternoon Cohort	Session 1 (PM, repeated)	Session 2 (PM, repeated)	Session 3 (PM, repeated)	Session 4 (PM, repeated)

Each cohort attends one slot per day (AM or PM) and follows Session 1 through Session 4 across the four days, a 4-day course for that cohort. The instructor delivers the same session content twice each day, once for the AM cohort and once for the PM cohort.

Course Overview

This course teaches the fundamentals of API design and development: what APIs are and why they matter, how REST/SOAP/RPC protocols differ, how to write and document API contracts, how to manage an API's lifecycle and secure it, and how APIs fit into a modern DevOps ecosystem. Hands-on labs use a Spring Boot REST API as the shared example, with Postman for exploring/testing endpoints, JUnit5 for automated tests, and IntelliJ as the development environment throughout.

Main Topics Covered

- APIs: Purpose, Nomenclature & Types
- Benefits and drawbacks of APIs, key terms, API types (Open, Partner, Internal)
- API Protocols & RESTful Basics
- REST, SOAP, RPC overview; standards (HTTP methods, status codes, resource naming)
- API Contracts & Documentation
- OpenAPI/Swagger standards; documenting APIs and common tooling
- API Lifecycle & Versioning
- Common lifecycle approaches, pros/cons, non-breaking version strategies
- Security for APIs
- Authentication, authorization, rate limiting
- DevOps & the API Ecosystem
- API First strategy, CI/CD, automated testing, security within the ecosystem

Learning Outcomes

- Explain the purpose of APIs and define key terms in API development
- Differentiate between API types and API protocols
- Build contracts into APIs and describe RESTful standards
- Discuss the API lifecycle and design a non-breaking versioning strategy
- Understand why security matters for APIs and configure basic security rules
- Describe how APIs fit into a DevOps/CI/CD ecosystem

Project Context

Labs build on a shared Spring Boot REST API example, extended lab by lab. You'll work in IntelliJ as the IDE, explore and test endpoints in Postman, write automated tests in JUnit5, and use a Swagger/OpenAPI editor for the contract and documentation labs.

Day 1 (4 Hours): API Purpose, Types & Protocols

Delivered once for the morning cohort and once (repeated) for the afternoon cohort.

1. Welcome, Learner Introductions & Course Objectives (15 min)

Course arc preview: what's covered across the 4 days.

2. APIs: Purpose, Nomenclature & Types (35 min)

- APIs: purpose and nomenclature
- Benefits and drawbacks of APIs
- Key terms in API development
- API types: Open, Partner, Internal

Kahoot 1 (10 min)

Bio Break (10 min)

3. API Protocols Overview: REST, SOAP & RPC (30 min)

Bio Break (10 min)

4. RESTful APIs: The Basics (35 min)

- Drawbacks of not having standards
- Common standards: HTTP methods, status codes, resource naming

5. Breakout 1: Lab 1.1: Analyzing an Existing API (30 min)

- Analyze an existing API to identify its protocol and standards adherence
- Stretch goals: compare a second API against the same checklist; document where it deviates from REST conventions

Bio Break (10 min)

Kahoot 2 (10 min)

6. Discussion & Buffer: API Types/Protocols Deep-Dive, Q&A (35 min)

- Open discussion on API types and protocol trade-offs from Lab 1.1
- Flex time: catch-up room for cohorts running behind, or extended Lab 1.1 stretch goals

7. Day 1 Wrap-up & Day 2 Preview (10 min)

Day 1 Deliverable: Working understanding of API purpose, types, and protocols; a completed protocol/standards analysis of an existing API (Lab 1.1).

Day 2 (4 Hours): API Contracts, Documentation & Design

Delivered once for the morning cohort and once (repeated) for the afternoon cohort. Continuation of Day 1 for the same cohort.

1. Day 2 Kickoff & Day 1 Recap (10 min)

2. API Contracts: Standards & OpenAPI/Swagger (35 min)

- Drawbacks of not having standards
- Summary of common standards (OpenAPI/Swagger)

Kahoot 1 (10 min)

Bio Break (10 min)

3. Breakout 1: Lab 1.2: Writing an OpenAPI Specification (30 min)

- Write an OpenAPI specification for a basic resource
- Stretch goals: add request/response examples and error schemas

Bio Break (10 min)

4. Documenting APIs: Standards & Tools (35 min)

- Drawbacks of not having standards
- Summary of common standards and tools

5. Breakout 2: Lab 1.3: Peer Review & Interactive Docs (30 min)

- Peer review of API contracts
- Generate interactive documentation (Swagger UI)
- Stretch goals: annotate edge cases; add auth documentation to the spec

Bio Break (10 min)

Kahoot 2 (10 min)

6. Design Lab Discussion: Contract-First vs. Code-First (40 min)

- Comparing contract-first and code-first workflows using Postman/Swagger tooling
- Group review of Lab 1.2/1.3 outputs

7. Day 2 Wrap-up & Day 3 Preview (10 min)

Day 2 Deliverable: An OpenAPI specification for a basic resource, peer-reviewed, with generated interactive documentation (Labs 1.2-1.3).

Day 3 (4 Hours): API Lifecycle & Security

Delivered once for the morning cohort and once (repeated) for the afternoon cohort. Continuation of Day 2 for the same cohort.

1. Day 3 Kickoff & Day 2 Recap (10 min)

2. API Lifecycle: Overview & Versioning (35 min)

- API lifecycle overview
- API versioning
- Review of most common approaches to lifecycle (pros and cons)

Kahoot 1 (10 min)

Bio Break (10 min)

3. Breakout 1: Lab 2.1: Designing a Non-Breaking Version Strategy (30 min)

- Design a non-breaking version strategy for an evolving API
- Stretch goals: draft a deprecation/sunset policy; handle a breaking-change edge case

Bio Break (10 min)

4. Security for APIs: Standards (35 min)

- Drawbacks of not having standards
- Common standards: authentication, authorization, rate limiting

5. Breakout 2: Lab 2.2: Implementing Basic API Security Rules (30 min)

- Implement or configure basic API security rules on the Spring Boot API
- Stretch goals: add rate-limit tiers; validate auth flows with a Postman collection

Bio Break (10 min)

Kahoot 2 (10 min)

6. Lifecycle & Security Standards Deep-Dive, Q&A (40 min)

- Drawbacks of skipping standards for both lifecycle and security, with real-world examples
- Group review of Lab 2.1/2.2 outputs

7. Day 3 Wrap-up & Day 4 Preview (10 min)

Day 3 Deliverable: A non-breaking versioning strategy (Lab 2.1) and configured basic security rules (Lab 2.2) on the shared API.

Day 4 (4 Hours): DevOps, API Ecosystem & Capstone

Delivered once for the morning cohort and once (repeated) for the afternoon cohort. Continuation of Day 3 for the same cohort.

1. Day 4 Kickoff & Day 3 Recap (10 min)

2. DevOps in API Development: Overview (30 min)

Kahoot 1 (10 min)

Bio Break (10 min)

3. API Ecosystem (35 min)

- API First strategy
- DevOps (CI/CD, automated testing)
- API security within the ecosystem
- The case for deep knowledge

Bio Break (10 min)

4. Breakout 1: Lab 2.3: Simulating an Automated API Test Suite (30 min)

- Build a JUnit5 test suite for the Spring Boot API, run from IntelliJ
- Stretch goals: add negative test cases; integrate a Postman collection runner

Kahoot 2 (10 min)

Bio Break (10 min)

5. Breakout 2: Lab 2.4: Capstone Exercise (40 min)

Capstone exercise integrating design, documentation, versioning, security, and testing from Days 1-4.

6. Review of Capstone Solutions & Discussion (35 min)

7. Course Recap, Final Q&A & Next Steps (10 min)

Day 4 Deliverable: Completed capstone integrating contract design, documentation, versioning, security, and an automated JUnit5 test suite.

🕒 Session Breakdown Table

Day 1 (4 hours / 240 min)

Topic	Duration
Welcome, Learner Introductions & Course Objectives	15 min

Topic	Duration
APIs: Purpose, Nomenclature & Types	35 min
Kahoot 1	10 min
Bio Break	10 min
API Protocols Overview: REST, SOAP & RPC	30 min
Bio Break	10 min
RESTful APIs: The Basics	35 min
Breakout 1: Lab 1.1: Analyzing an Existing API	30 min
Bio Break	10 min
Kahoot 2	10 min
Discussion & Buffer: API Types/Protocols Deep-Dive, Q&A	35 min
Day 1 Wrap-up & Day 2 Preview	10 min
Total	**240 min**

Day 2 (4 hours / 240 min)

Topic	Duration
Day 2 Kickoff & Day 1 Recap	10 min
API Contracts: Standards & OpenAPI/Swagger	35 min
Kahoot 1	10 min
Bio Break	10 min
Breakout 1: Lab 1.2: Writing an OpenAPI Specification	30 min
Bio Break	10 min
Documenting APIs: Standards & Tools	35 min
Breakout 2: Lab 1.3: Peer Review & Interactive Docs	30 min
Bio Break	10 min
Kahoot 2	10 min
Design Lab Discussion: Contract-First vs. Code-First	40 min
Day 2 Wrap-up & Day 3 Preview	10 min
Total	**240 min**

Day 3 (4 hours / 240 min)

Topic	Duration
Day 3 Kickoff & Day 2 Recap	10 min
API Lifecycle: Overview & Versioning	35 min
Kahoot 1	10 min

Topic	Duration
Bio Break	10 min
Breakout 1: Lab 2.1: Designing a Non-Breaking Version Strategy	30 min
Bio Break	10 min
Security for APIs: Standards	35 min
Breakout 2: Lab 2.2: Implementing Basic API Security Rules	30 min
Bio Break	10 min
Kahoot 2	10 min
Lifecycle & Security Standards Deep-Dive, Q&A	40 min
Day 3 Wrap-up & Day 4 Preview	10 min
Total	**240 min**

Day 4 (4 hours / 240 min)

Topic	Duration
Day 4 Kickoff & Day 3 Recap	10 min
DevOps in API Development: Overview	30 min
Kahoot 1	10 min
Bio Break	10 min
API Ecosystem	35 min
Bio Break	10 min
Breakout 1: Lab 2.3: Simulating an Automated API Test Suite	30 min
Kahoot 2	10 min
Bio Break	10 min
Breakout 2: Lab 2.4: Capstone Exercise	40 min
Review of Capstone Solutions & Discussion	35 min
Course Recap, Final Q&A & Next Steps	10 min
Total	**240 min**

Combined Total Duration: 16 hours (960 minutes) across 4 sessions / 4 days, run twice per day for 2 cohorts.

✅ Deliverables (End of Course)

- Protocol/standards analysis of an existing API (Lab 1.1)
- OpenAPI specification for a basic resource, peer-reviewed, with interactive docs (Labs 1.2-1.3)

- Non-breaking versioning strategy (Lab 2.1)
- Configured basic API security rules (Lab 2.2)
- JUnit5 automated test suite for the shared Spring Boot API (Lab 2.3)
- Capstone integrating design, documentation, versioning, security, and testing (Lab 2.4)

Teaching Philosophy

The course uses one Spring Boot API example throughout, so each lab builds on the last, tested with Postman and JUnit5 in IntelliJ.

Course Schedule

Day 1: API Purpose, Types & Protocols

Understanding what APIs are, why they matter, and the core protocols that power them.

- Concepts: Key API nomenclature, benefits/drawbacks, and API types (Open, Partner, Internal).
- Protocols: High-level overview of REST, SOAP, and RPC.
- RESTful Basics: HTTP methods, status codes, and resource naming conventions.
- Hands-on: Analyzing an existing API to identify its protocol and adherence to standards.

Day 2: API Contracts, Documentation & Design

Designing robust API contracts and documenting them effectively.

- Contracts: Introduction to OpenAPI/Swagger standards and their importance.
- Documentation: Exploring standard tools for generating interactive documentation, such as Swagger UI.
- Design Workflows: Comparing contract-first versus code-first approaches.
- Hands-on: Writing an OpenAPI specification from scratch and generating interactive docs for peer review.

Day 3: API Lifecycle & Security

Managing the evolution of APIs over time and securing them against common threats.

- Lifecycle Management: Strategies for versioning, handling non-breaking changes, deprecation, and sunsetting.
- Security Standards: Authentication, authorization, and rate limiting.
- Hands-on: Designing a non-breaking version strategy for an evolving API and implementing basic security rules.

Day 4: DevOps, API Ecosystem & Capstone

Integrating APIs into a modern DevOps ecosystem and applying all learned concepts.

- DevOps & Ecosystem: Understanding API-First strategies, CI/CD pipelines, and automated testing.
- Automated Testing: Simulating an automated API test suite using JUnit5.
- Hands-on Capstone: A comprehensive final exercise integrating contract design, documentation, versioning, security, and testing.

API Dev Quickstart

A minimal Spring Boot REST API used to verify your installation for the **Fundamentals of API Development** course. It doesn't teach any course content by itself; it's just a fast way to confirm that Java, Maven, IntelliJ IDEA (or VS Code), and Postman are all working together before Day 1.

Prerequisites

Complete the macOS or Windows install guide first.

Open in IntelliJ IDEA

1. Launch IntelliJ IDEA.
2. **File > Open...** and select the `quickstart-project` folder.
3. IntelliJ should detect `pom.xml` and import it as a Maven project automatically (this can take a minute the first time while it downloads dependencies).

Run from the Command Line

```
cd quickstart-project
mvn clean compile
mvn spring-boot:run
```

The app starts on port 8080. Leave it running and move to the "Verify with Postman" section below. Stop it anytime with `Ctrl+C`.

Open in VS Code

1. Launch VS Code.
2. **File > Open Folder...** and select the `quickstart-project` folder.
3. VS Code will detect `pom.xml` and prompt you to import the Maven project. Accept it.

Run from IntelliJ IDEA

Open `src/main/java/com/apidev/quickstart/QuickstartApplication.java` and click the green run arrow next to `main`.

Verify with a Browser or curl

```
curl http://localhost:8080/api/v1/health
curl http://localhost:8080/api/v1/greetings/YourName
```

You should see:

```
{"status":"UP","service":"api-dev-quickstart"}
{"message":"Hello, YourName! Your API dev setup is working."}
```

API Documentation (Swagger UI & OpenAPI)

This project includes `springdoc-openapi` 2.5.0, which automatically generates an OpenAPI 3.0 specification from your controllers at runtime. No separate generation step is needed — just start the app.

View the Interactive Docs

Renderer	URL	Style
Swagger UI	`http://localhost:8080/swagger-ui.html`	Interactive "try it out" console
Redoc	`http://localhost:8080/redoc.html`	Clean three-panel reference layout

Both render the same OpenAPI spec served at `/api-docs`. The `redoc.html` page is included in `src/main/resources/static/`.

Fetch the Raw Spec

```
# JSON format
curl http://localhost:8080/api-docs

# YAML format
curl http://localhost:8080/api-docs.yaml

# Save to a file
curl http://localhost:8080/api-docs -o openapi.json
```

How It Works

`springdoc-openapi-starter-webmvc-ui` introspects your `@RestController` classes on startup and serves the generated spec at `/api-docs` (JSON) and `/swagger-ui.html` (interactive console). These paths are configured in `src/main/resources/application.properties`:

```
springdoc.api-docs.path=/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
```

To enrich the generated spec with descriptions, examples, and error responses, add `@Operation`, `@ApiResponse`, and `@Parameter` annotations to your controller methods. See the course slides and `api-dev-companion/day2/concepts/springdoc-openapi.md` for the full annotation reference.

Verify with Postman

1. Open Postman.
2. **Import** the collection at `postman/api-dev-quickstart.postman_collection.json`.
3. With the app running, click **Run** on the collection (or send each request individually).
4. All three requests, `Health Check`, `Greeting`, and `OpenAPI Docs`, should return status `200` and pass their built-in tests.

Run the JUnit5 Tests

From the command line:

```
mvn test
```

Or in IntelliJ or VS Code: right-click the `src/test/java` folder and choose **Run 'All Tests'**.

Both test classes (`HealthControllerTest`, `GreetingControllerTest`) should pass.

Project Structure

```
quickstart-project/
├── pom.xml
├── postman/
|   └── api-dev-quickstart.postman_collection.json
└── src/
    ├── main/
    |   ├── java/com/apidev/quickstart/
    |   |   ├── QuickstartApplication.java
    |   |   └── controller/
    |   |       ├── HealthController.java
    |   |       └── GreetingController.java
    |   └── resources/application.properties
    └── test/
        └── java/com/apidev/quickstart/controller/
```

```
            ├── HealthControllerTest.java
            └── GreetingControllerTest.java
```

Troubleshooting

- **Port 8080 already in use**: free up the port (see below), or run with a different port: `mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8081` (update the Postman `baseUrl` variable to match).
- **Maven can't resolve dependencies**: check your internet connection, or see the proxy/ `settings.xml` notes in the install guides.
- **IntelliJ shows Maven import errors**: right-click `pom.xml` and choose **Maven > Reload project**.

Free Up Port 8080

Useful any time port 8080 is already taken, not just during this quickstart, for example if a previous `mvn spring-boot:run` was left running or another app grabbed the port.

macOS / Linux:

```
lsof -ti:8080 | xargs kill -9
```

Windows (Command Prompt):

```
netstat -ano | findstr :8080
taskkill /PID <PID> /F
```

(Replace `<PID>` with the process ID from the last column of the `netstat` output.)

Windows (PowerShell):

```
Get-Process -Id (Get-NetTCPConnection -LocalPort 8080).OwningProcess |
Stop-Process -Force
```

APIs: Purpose, Nomenclature and Types

Reference for what an API is, the vocabulary this course uses, what publishing one buys you, and what it costs.

Learning Objectives

- *Define an API as a contract between provider and consumer and explain what that contract buys*
- *Distinguish the two independent axes every API sits on: audience and protocol*
- *Classify an API by its audience type and describe the trade-offs of each*
- *Use the course vocabulary consistently when reading specs, writing code, or reviewing a peer's work*
- *Identify the recurring costs and risks that come with publishing any API*

Core Idea

An API is a contract between a provider and a consumer. The provider promises that a specific request will produce a specific response. The consumer writes code against that promise and against nothing else. Everything valuable about APIs and everything painful about them follows from that single sentence: because the consumer depends only on the contract, the provider can change the implementation freely, and because the consumer depends on the contract, the provider cannot change it freely.

Two Independent Axes

The phrase "type of API" answers two different questions, and mixing them causes most of the confusion in this area. Keep them separate.

Axis	Values	Question it answers
Audience	Open, Partner, Internal, Composite	Who may call it, under what agreement
Protocol	REST, SOAP, RPC, GraphQL, WebSocket	How the call is shaped on the wire

An API has one value on each axis. A partner API delivered over gRPC is perfectly ordinary. This document covers the audience axis; the protocol axis has its own reference.

Audience Types

Type	Consumer	Auth posture	Cost of a breaking change
Open	Anyone who signs up	API key or OAuth, public docs, published deprecation policy	Highest, because you cannot contact every caller
Partner	Named organizations under contract	OAuth client credentials or mutual TLS, SLA, negotiated windows	Moderate, because the list of callers is known
Internal	Teams inside your organization	Service identity, usually behind a gateway or mesh	Lowest per change, highest in aggregate
Composite	Usually an internal or partner caller	Inherits from the layer it fronts	Varies, since it bundles several downstream calls

> ⚠ **Warning**
> Internal APIs are where organizations under-invest, and it shows up as sprawl: hundreds or thousands of endpoints spread across several gateways with no single catalog. An endpoint nobody can list is an endpoint nobody can secure, version, or retire.

Vocabulary Used Throughout This Course

Term	Definition
Resource	The thing the API exposes, named by a noun, such as `/orders`
Endpoint	A method plus a path acting on a resource, such as `GET /api/v1/orders/42`
Contract	The machine-readable description of requests, responses, and errors
Provider and consumer	Who serves the API, and who calls it
Payload	The request or response body
Safe	The call does not change server state
Idempotent	Repeating the call leaves the same server state
Gateway	

Term	Definition
	The component fronting one or more APIs for auth, routing, and rate limiting

What You Gain

Decoupling comes first, because it enables the rest. A consumer that depends on `GET /api/v1/accounts/42` does not know or care whether the data comes from a relational table, a cache, or a third-party service, so the provider can replace any of that without a conversation. Reuse follows: one documented capability serves the web app, the mobile app, the nightly batch job, and a partner. Composability follows from reuse, and it is the point at which the API stops being plumbing and becomes the product surface.

> 💜 **Important**
>
> Two claims are worth resisting. APIs do not make systems simpler; they relocate complexity from a function call into a network hop and a contract. APIs are not faster than in-process calls; they are slower, and you accept that in exchange for independent deployment.

What You Owe

The contract becomes someone else's dependency, which means your change becomes their outage.

- Twitter's 2018 API changes broke Tweetbot and Twitterrific, produced a public backlash under `#breakingmytwitter`, and forced a delay.
- Google Maps Platform's repricing cut the free tier from roughly 750,000 to 25,000 calls per month, and consumers reported order-of-magnitude cost increases.
- Google's earlier API retirements, including the Translate API, drew hundreds of negative responses and a partial reversal.

You also owe the network: latency you did not have before, partial failures, and therefore timeouts, retries, and idempotency. And you owe an inventory, because sprawl is what accumulates when nobody is responsible for the list.

What Can Go Wrong

> ⚠️ **Common Pitfalls**
>
> The common failure is treating "API" as a synonym for "REST endpoint" and skipping the contract because the team is small. The contract is the deliverable; REST is one way to express it. Small teams grow into sprawl, and retrofitting a catalog onto four hundred undocumented endpoints costs far more than writing them down as they were built.

Chapter Summary

Key Concepts	Key Takeaways
API as a contract	A provider promises a specific response for a specific request; the consumer codes against that promise alone
Two independent axes	Every API has an audience type and a protocol; keep them separate or the discussion collapses
Audience types	Open, Partner, Internal, and Composite differ in who calls, how auth works, and what a breaking change costs
Course vocabulary	Resource, endpoint, contract, provider/consumer, payload, safe, idempotent, gateway. Use these terms consistently
What you gain	Decoupling, reuse, and composability; not simplicity or raw speed
What you owe	A dependency, a network, and an inventory. Skipping any one compounds over time

API Protocols: REST, SOAP and RPC

Reference for the three protocol families, how each shapes the same request on the wire, and how to choose between them per boundary.

Learning Objectives

- *Distinguish resource-oriented (REST) from operation-oriented (SOAP, RPC) protocols*
- *Read the same request in REST, SOAP, JSON-RPC, and gRPC shapes and identify where errors land in each*
- *Compare protocols across the dimensions that decide real projects: contract enforcement, error signalling, caching, and tooling*
- *Choose a protocol per boundary rather than per organization*

The One Distinction That Organizes Everything

REST is resource-oriented. The URL names a thing, the HTTP method names the verb, and the HTTP status code carries the outcome. SOAP and RPC are operation-oriented. The URL names a single endpoint, the body names the procedure to run, and the transport status says little about what happened. Caching, discoverability, tooling, and error handling all follow from this one difference, so it is worth getting right before anything else.

> ℹ️ **Note**
>
> REST is an architectural style layered over HTTP, not a protocol. SOAP is a protocol. This matters on Day 2, when the contract for a REST API turns out to be an optional external document (OpenAPI) while the contract for a SOAP service is mandatory and built in (WSDL).

The Same Operation in Four Shapes

Fetching account 42 looks like this in REST, where the method and path carry the whole request:

```
GET /api/v1/accounts/42 HTTP/1.1
Accept: application/json
```

In SOAP it is always a `POST` to one endpoint, with the operation named inside the envelope:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getAccount xmlns="http://bank.example/accounts">
      <accountId>42</accountId>
    </getAccount>
  </soap:Body>
</soap:Envelope>
```

In JSON-RPC 2.0 it is also a `POST` to one endpoint, with the operation in a `method` field:

```
{"jsonrpc": "2.0", "method": "getAccount", "params": {"accountId":
42}, "id": 1}
```

In gRPC the operation is a method on a service declared in a `.proto` file and serialized as Protobuf over HTTP/2:

```
service AccountService {
  rpc GetAccount (GetAccountRequest) returns (Account);
}
```

> 💜 **Important**
>
> The detail students find most surprising is where errors go. A missing account is `404` in REST. In SOAP it is HTTP `500` carrying a `soap:Fault`, so a business outcome arrives as a transport failure. In JSON-RPC it is HTTP `200` carrying an `error` member, so a failure arrives as a success. Any monitoring built on HTTP status codes reads those two cases wrong unless it is written specifically for them.

Comparison

Dimension	REST	SOAP	RPC (JSON-RPC, gRPC)
Organizing unit	Resource	Operation in an envelope	Procedure
Transport	HTTP, all methods	Usually HTTP POST	HTTP POST, or HTTP/2 for gRPC
Payload	Usually JSON	XML only	JSON or Protobuf
Contract	OpenAPI, external and optional	WSDL, built in and mandatory	`.proto`, mandatory for gRPC
Error signal	HTTP status code		

Dimension	REST	SOAP	RPC (JSON-RPC, gRPC)
		`soap:Fault` inside HTTP 500	Error object inside HTTP 200, or gRPC status
HTTP caching	Native for `GET`	No	No
Debuggable with curl	Yes	Awkwardly	Not for gRPC

Where Each One Wins in 2026

REST remains the default at the public edge. It carries roughly 83% of public APIs and around 89% enterprise usage, and it wins there because of native HTTP caching, universal tooling, and the fact that anyone can reproduce a bug with curl.

gRPC has become the standard for internal service-to-service traffic at organizations that care about throughput, with reported serialization gains of seven to ten times over JSON for heavy workloads. It is a poor fit for a public API that third parties consume from a browser.

GraphQL sits at roughly 25% enterprise adoption, down from a peak near 40%, and its durable pattern is backend-for-frontend: internal traffic stays REST or gRPC while external clients consume one graph.

SOAP persists in banking, insurance, government, and telecom, and not out of nostalgia. It offers three things REST does not provide out of the box:

- A machine-enforced contract with strict XML Schema typing
- Message-level security through WS-Security, so signatures travel with the message rather than terminating at TLS
- Transactional and reliable-messaging semantics through the WS-* stack

> 💡 **Tip**
> When an integration must be signed, typed, and non-repudiable across several hops, SOAP is the shortest path.

The practical consequence is that protocol choice is a per-boundary decision, not a company-wide policy. The common 2026 architecture is REST at the public edge, GraphQL or a typed RPC layer for internal frontends, and gRPC between services.

What Can Go Wrong

> ⚠️ **Common Pitfalls**
>
> Two mistakes recur. The first is declaring SOAP dead, which surprises the student who maintains one next month. The second is comparing protocols on payload size alone, when contract enforcement, security model, and available tooling are what actually decide real projects.

Chapter Summary

Key Concepts	Key Takeaways
Resource vs. operation orientation	REST names things; SOAP and RPC name procedures. Caching, tooling, and error handling all follow from this one distinction
Error signalling	A `404` in REST is a `500` with a `soap:Fault` in SOAP and a `200` with an error object in JSON-RPC. Monitoring must account for each
Protocol comparison	Compare on contract enforcement, error model, caching, and tooling, not payload size alone
Per-boundary choice	REST at the public edge, gRPC between services, GraphQL for internal frontends, SOAP where signatures and transactions are non-negotiable

RESTful Standards: Methods, Status Codes and Resource Naming

Reference for the REST conventions this course holds APIs to, and the checklist Lab 1.1 scores against.

Learning Objectives

- *Explain why REST conventions exist: so generic machinery (caches, proxies, gateways) works without per-API custom code*
- *Map each HTTP method to its safety, idempotency, and correct success code*
- *Distinguish the status code families and the high-stakes distinctions within them*
- *Apply the eight resource naming rules and explain which defects each one prevents*
- *Score an existing API against the Lab 1.1 checklist*

Why Standards Instead of Preferences

Conventions in REST exist so that generic machinery works. A cache can serve a `GET` because `GET` is defined as safe. A proxy can retry a `PUT` because `PUT` is defined as idempotent. A gateway can alert on error rates because errors are defined as 4xx and 5xx. Break any of those definitions and the generic machinery silently does the wrong thing, and every consumer has to write bespoke handling for your API specifically.

> 💜 **Important**
> That is the real cost of skipping standards, and it compounds. Onboarding cost becomes per-API instead of per-organization. Documentation drifts, because nothing about a URL like `/doStuff` is self-describing. Security review restarts from zero for every service. With most API traffic in 2026 coming from programs rather than people, predictable status codes and machine-readable errors matter more than a friendly message inside a `200`.

Methods and Their Guarantees

Method	Safe	Idempotent	Use	Success code
GET	yes	yes	Read a resource or collection	200
HEAD	yes	yes	Metadata only, no body	200
POST	no	no	Create a subordinate resource, or a non-CRUD action	201 with `Location`, or 200
PUT	no	yes	Replace the resource at a known URI	200 or 204
PATCH	no	no	Partial update	200 or 204
DELETE	no	yes	Remove a resource	204, or 200 with a body

Safe means the call does not change server state. Idempotent means repeating it leaves the same state.

> 🛑 **Caution**
> A `GET` that mutates and a `DELETE` that returns `500` on the second call are the two violations that produce the most mysterious production behaviour, because retries and caches are built on the assumption that neither happens.

Status Codes Worth Knowing

Learn the families first, then a short list inside each.

Family	Meaning for the caller	Codes to know
2xx	It worked	200 OK, 201 Created, 202 Accepted, 204 No Content
3xx	Look elsewhere, or use your cache	301 Moved Permanently, 304 Not Modified
4xx	Your request was wrong, do not retry it unchanged	400, 401, 403, 404, 405, 409, 422, 429
5xx	Something failed on my side, a retry may help	500, 502, 503, 504

Three distinctions account for most mistakes.

- `401` means the request is unauthenticated, so the server does not know who is calling; `403` means the caller is known and not permitted.

- `400` means the server could not parse the request; `422` means it parsed fine and failed semantic validation.
- Returning `200 OK` with a body like `{"success": false}` is the single most damaging shortcut on this list, because it defeats every retry policy, every gateway metric, every cache, and every generic client at once.

> ⚠️ **Warning**
> Returning `200 OK` with `{"success": false}` defeats every retry policy, every gateway metric, every cache, and every generic client at once. It is the single most damaging shortcut on this list.

Error Bodies Have a Standard Now

RFC 9457, Problem Details for HTTP APIs, replaces RFC 7807 and defines the error shape so consumers stop guessing. It is served as `application/problem+json` and carries `type`, `title`, `status`, `detail`, and `instance`, plus any extensions you need.

```
{
  "type": "https://api.example.com/problems/insufficient-funds",
  "title": "Insufficient funds",
  "status": 409,
  "detail": "Account 42 has a balance of 12.50 and the transfer
requires 100.00",
  "instance": "/api/v1/accounts/42/transfers/8871"
}
```

> ℹ️ **Note**
> Spring has supported this since Framework 6.0 through the `ProblemDetail` type, opted into per controller advice. Spring Boot 4 makes it a framework-wide default instead. This course runs on Spring Boot 3.2.5, so the opt-in form is what you will write.

Resource Naming Rules

These are ordered by how many defects each one catches.

1. **Nouns, not verbs.** Use `/orders`, never `/getOrders` or `/createOrder`. The verb is already the HTTP method.
2. **Plural collections.** Use `/accounts/42`, not `/account/42`. One rule applied consistently beats a mixture.

3. **Hierarchy expresses containment.** `/customers/17/orders` reads as the orders belonging to customer 17.
4. **Lowercase with hyphens for multi-word segments.** Use `/purchase-orders`, not `/purchaseOrders` or `/purchase_orders`.
5. **The path identifies, the query filters and paginates.** Use `/orders?status=shipped&page=2&size=25`, not `/orders/shipped`.
6. **No file extensions and no trailing slash.** Content negotiation belongs in the `Accept` header.
7. **Version explicitly.** `/api/v1/orders` is the pragmatic default and the one this course's example API uses. Day 3 covers header and media-type versioning and their trade-offs.
8. **Do not leak the implementation.** A path like `/api/v1/tbl_cust_master` publishes your schema and guarantees that a refactor becomes a breaking change.

What Lab 1.1 Checks

The lab checklist is this document turned into questions:

- Identify the protocol
- Score resource naming against the eight rules
- Check method usage against the safe and idempotent guarantees
- Check status code correctness, with attention to `401` against `403`, `400` against `422`, and any `200` carrying an error
- Score the error body against RFC 9457
- Check the versioning approach
- Check whether documentation exists at all

Chapter Summary

Key Concepts	Key Takeaways
Standards enable generic machinery	Caches, proxies, and gateways depend on method semantics and status code families. Break them and every consumer writes bespoke handling
Method guarantees	Safe means no state change; idempotent means repeatable. `GET` mutations and non-idempotent `DELETE` cause the hardest production bugs

Key Concepts	Key Takeaways
Status code families	2xx success, 3xx redirect, 4xx client error (do not retry unchanged), 5xx server error (retry may help)
High-stakes distinctions	401 vs 403, 400 vs 422, and never bury an error inside a 200
RFC 9457 error bodies	Standard shape with `type`, `title`, `status`, `detail`, and `instance`. Spring supports it via `ProblemDetail`
Resource naming	Eight ordered rules, from nouns-not-verbs to don't-leak-the-schema

API Contracts: Standards & OpenAPI/Swagger

Reference for what an API contract is, why it needs a standard shape, and how OpenAPI defines that shape. Covers what Lab 1.2 asks you to write.

Learning Objectives

- *Define an API contract as the formal agreement between provider and consumer about what a call sends and returns*
- *Explain why a standard machine-readable format beats a free-form document*
- *Identify the four essential sections of an OpenAPI 3.1 document*
- *Write a minimal OpenAPI spec with* `info`, `servers`, `paths`, *and* `components/schemas`
- *Avoid the three most common mistakes: spec-after-code drift, blank descriptions, and Swagger 2.0 confusion*

Core Idea

An API contract is the formal agreement between a provider and a consumer about what a call sends and what it returns. Without a written contract, that agreement lives in email threads, tribal memory, and whatever the code happens to do this week. OpenAPI is the standard shape for writing the agreement down so both sides, and the tooling both sides depend on, read the same thing.

Why Standards Instead of a Free-Form Doc

> 💜 **Important**
>
> A hand-written description of an API drifts, because nothing forces it to match the code, and every reader has to interpret prose differently. A standard machine-readable format fixes both problems: it can be validated, rendered into interactive docs, and fed to code generators, none of which work on a paragraph of English. The cost of skipping this standard scales with the number of consumers, since each one re-derives the contract by trial and error against a running server instead of reading one file.

Anatomy of an OpenAPI Document

The current stable line is OpenAPI 3.1, with 3.2 as the newest feature release. Every document has four parts worth knowing cold.

Section	Holds	Example
info	Title, version, contact	`title: Accounts API`, `version: 1.0.0`
servers	Base URLs the API actually runs on	`https://api.example.com/api/v1`
paths	Every endpoint, its methods, parameters, and responses	`/orders/{id}: get: ...`
components/ schemas	Reusable data shapes referenced from any path	`Order`, `Customer`

```yaml
openapi: 3.1.0
info:
  title: Accounts API
  version: 1.0.0
paths:
  /accounts/{id}:
    get:
      summary: Get an account by id
      responses:
        '200':
          description: Account found
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Account'
```

> 💡 **Tip**
>
> A schema defined once in `components` and referenced by `$ref` from every path that needs it keeps the naming and shape consistent across the whole document, which is exactly what a reviewer checks in Lab 1.3.

What Can Go Wrong

> ⚠️ **Common Pitfalls**
>
> The most common failure is writing the spec after the code, then letting the two drift apart, which defeats the entire point of writing a contract first. A close second is leaving `description` fields blank throughout; every linter tolerates it, but it is what makes generated docs unusable. A third is copying an old `swagger: "2.0"` example instead of the current `openapi: "3.1.0"` root field. Swagger 2.0 is a different, older format, and the two are not interchangeable.

What Lab 1.2 Checks

The lab asks you to write an OpenAPI 3.1 specification for a basic resource from scratch, so the checklist is this document turned into requirements: a complete `info` block, an accurate `servers` entry, at least one path with a described request and response, and a `components/schemas` entry referenced by `$ref` rather than repeated inline.

Chapter Summary

Key Concepts	Key Takeaways
Contract as the source of truth	A machine-readable spec is the single agreement both sides and their tooling read from
OpenAPI anatomy	`info`, `servers`, `paths`, and `components/schemas`. Know all four
`$ref` for consistency	Define schemas once in `components` and reference them; never repeat inline
Common mistakes	Spec-after-code drift, blank descriptions, confusing Swagger 2.0 with OpenAPI 3.1

Documenting APIs: Standards & Tools

Reference for how an OpenAPI document becomes interactive documentation, and what a peer review of that documentation should check. Covers what Lab 1.3 asks you to do.

Learning Objectives

- *Explain why generated docs stay in sync with the spec by construction*
- *Trace the pipeline from Spring annotations through springdoc-openapi to Swagger UI*
- *Compare Swagger UI (interactive console) and Redoc (reference layout) and know when to use each*
- *Conduct a peer review of an API contract that checks descriptions, error responses, and live render accuracy*

Core Idea

Documentation generated from the spec stays in sync with the spec by construction, because it has no separate copy to drift. A wiki page describing an endpoint has to be updated by a person who remembers to do it. A doc page rendered from the OpenAPI file updates the moment the file changes. This is why the tooling in this session reads the spec directly instead of maintaining prose alongside it.

From Annotations to a Live Console

On this course's Spring Boot API, the document is generated at runtime rather than hand-maintained. `springdoc-openapi` inspects your controllers and their annotations and produces the OpenAPI JSON automatically.

Step	What happens
Annotate	`@Operation`, `@Schema`, and standard Spring MVC annotations describe each endpoint
Introspect	springdoc-openapi reads the annotations and class structure at startup
Serve the spec	The generated document is served at `/api-docs`, a path this course customizes in `application.properties` (the library default is `/v3/api-docs`)
Render	Swagger UI reads that document and renders it at `/swagger-ui.html`

> ⚠️ **Warning**
> This course pins `springdoc-openapi` to 2.5.0 in the quickstart API's `pom.xml`, part of the 2.x line that targets Spring Boot 3.x; the 3.x line of the library only targets Spring Boot 4. Using the wrong line against Spring Boot 3.2.5 will simply fail to start, so check the version in `pom.xml` before assuming a fresh tutorial applies.

Swagger UI vs Redoc

Tool	Strength	When to reach for it
Swagger UI	Interactive "try it out" console	Exploring and testing endpoints during development, this course's default
Redoc	Clean three-panel reference layout	Public-facing docs where readability matters more than in-page testing

Both read the same OpenAPI document, so switching renderers never means rewriting the contract.

Peer Review as a Contract Review

Reviewing an API contract works the same way as reviewing code: it is a pass over the spec file, not a conversation about vibes. A useful pass checks whether every endpoint has a description, whether every schema field has a description and an example, and whether every realistic non-2xx response is documented and not just the happy path.

> 🛑 **Caution**
> Docs that only show a `200` response are technically complete and practically dangerous, since they hide exactly the error handling a consumer needs to plan for.

What Lab 1.3 Checks

Lab 1.3 has you peer review a partner's Lab 1.2 contract, then view it live through Swagger UI. The checklist is the same one above: complete descriptions, documented error responses, and a working interactive console that matches what was written in the spec, not a stale render from an earlier version of it.

Chapter Summary

Key Concepts	Key Takeaways
Generated docs don't drift	Docs rendered from the OpenAPI file update the moment the file changes. No separate copy to maintain
Annotation-to-console pipeline	`@Operation` → springdoc-openapi → `/api-docs` → Swagger UI at `/swagger-ui.html`
Swagger UI vs Redoc	Interactive console for development; clean reference layout for public-facing docs. Same spec, different renderer
Peer review checklist	Every endpoint described, every schema field described with an example, every non-2xx response documented

Springdoc-OpenAPI: Generating API Documentation

What It Does

`springdoc-openapi` is a library that automatically generates an OpenAPI 3.0 specification from your Spring Boot controllers at runtime. It introspects your `@RestController` classes, reads Spring MVC annotations (`@GetMapping` , `@PathVariable` , etc.), and produces a live spec — no separate build step required.

Adding It to Any Spring Boot Project

Add a single dependency to `pom.xml` :

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.5.0</version>
</dependency>
```

This one artifact includes both the OpenAPI JSON generator and the Swagger UI interactive console. No other dependencies are needed.

Version compatibility note: The 2.x line of springdoc-openapi targets Spring Boot 3.x. The 3.x line targets Spring Boot 4.x. This course uses Spring Boot 3.2.5 with springdoc-openapi 2.5.0. Mixing them the wrong way causes startup failures.

Generating and Accessing the Docs

Springdoc generates the spec at runtime — no separate `mvn generate` command is needed.

Step 1: Start the application

```
mvn spring-boot:run
```

Step 2: Access the docs

What	Default URL	What you get
OpenAPI JSON	`http://localhost:8080/v3/api-docs`	Machine-readable spec (JSON)
		Same spec in YAML

What	Default URL	What you get
OpenAPI YAML	`http://localhost:8080/v3/api-docs.yaml`	
Swagger UI	`http://localhost:8080/swagger-ui.html`	Interactive "try it out" console

In this course's quickstart API, the paths are customized in `application.properties`:

```
springdoc.api-docs.path=/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
```

Step 3: Export the spec as a file

```
# JSON
curl http://localhost:8080/api-docs -o openapi.json

# YAML
curl http://localhost:8080/api-docs.yaml -o openapi.yaml
```

What You Get Without Any Annotations

Even if your controllers carry no OpenAPI annotations at all, springdoc still generates a functional spec. It infers:

- Paths and HTTP methods from `@GetMapping`, `@PostMapping`, etc.
- Path parameters from `@PathVariable`
- Query parameters from `@RequestParam`
- Request bodies from `@RequestBody`

The spec will be correct but sparse — no descriptions, no examples, no error response documentation.

Enriching the Spec with Annotations

Add these annotations from `io.swagger.v3.oas.annotations` to your controllers:

`@Operation` — describe what an endpoint does

```
@Operation(
    summary = "Return service health status",
    description = "Returns a simple UP/DOWN status. Used by load
balancers and monitoring tools."
)
```

```
@GetMapping("/api/v1/health")
public Map<String, String> health() { ... }
```

`@ApiResponse` — document every realistic response code

```
@Operation(summary = "Get a personalized greeting")
@ApiResponse(
    responseCode = "200",
    description = "Greeting returned successfully",
    content = @Content(
        mediaType = "application/json",
        examples = @ExampleObject(value =
"{\"message\":\"Hello, Alice! Your API dev setup is working.\"}")
    )
)
@GetMapping("/api/v1/greetings/{name}")
public Map<String, String> greet(@PathVariable String name) { ... }
```

`@Parameter` — describe path and query parameters

```
@GetMapping("/api/v1/greetings/{name}")
public Map<String, String> greet(
    @Parameter(description = "Name to include in the greeting", require
d = true, example = "Alice")
    @PathVariable String name
) { ... }
```

`@Schema` — describe model fields

```
@Schema(description = "The greeting message returned to the caller",
example = "Hello, Alice!")
private String message;
```

Where to Find More Configuration

Property	Default	Purpose
`springdoc.api-docs.path`	`/v3/api-docs`	Where the OpenAPI JSON spec is served
`springdoc.swagger-ui.path`	`/swagger-ui.html`	Where the Swagger UI console is served
`springdoc.api-docs.enabled`	`true`	Set to `false` to disable spec generation entirely
`springdoc.swagger-ui.enabled`	`true`	Set to `false` to disable only the UI console

Add these to `src/main/resources/application.properties`.

Swagger UI vs. Redoc

Both tools render the same OpenAPI spec. They do not require different specs or annotations.

Tool	Strength	Best for
Swagger UI	Interactive "try it out" console	Exploring and testing during development
Redoc	Clean three-panel reference layout	Public-facing docs, readability first

Swagger UI is included with the `springdoc-openapi-starter-webmvc-ui` dependency. Redoc is not bundled, but you can access it two ways:

CDN (zero setup): Paste this URL while the API is running:

```
https://redocly.github.io/redoc/?url=http://localhost:8080/api-docs
```

Static page: Drop a `src/main/resources/static/redoc.html` file:

```
<!DOCTYPE html>
<html>
<head><title>API Docs - Redoc</title><meta charset="utf-8"/></head>
<body>
  <redoc spec-url="/api-docs"></redoc>
  <script src="https://cdn.redoc.ly/redoc/latest/bundles/
redoc.standalone.js"></script>
</body>
</html>
```

Then visit `http://localhost:8080/redoc.html` after restarting the app.

Both approaches consume the same `/api-docs` JSON. No annotation or configuration changes needed.

The Contract-First vs. Code-First Tradeoff

Approach	How the spec is created	Docs drift risk	Team parallelism
Contract-first	Hand-author the YAML spec, then implement	Possible (spec must be kept in sync)	High (frontend and backend build simultaneously)
Code-first	Write code, let springdoc generate the spec	None (spec regenerates on every run)	Low (frontend waits for code or a stub)

Lab 1.3 lets you experience both: your hand-written Lab 1.2 spec is contract-first; the annotated controllers produce a code-first spec via

springdoc. Compare them side by side to understand where each approach shines.

Troubleshooting

- **Swagger UI shows a blank page:** Confirm the app started without errors. Check `http://localhost:8080/api-docs` — if it returns JSON, the spec generation is working and the UI issue is likely a browser cache problem.
- **Annotations not appearing in the spec:** Did you restart the app after adding annotations? If using devtools, a hot reload may not pick up new annotations — do a full restart (`Ctrl+C` then `mvn spring-boot:run`).
- **Startup fails with "NoSuchMethodError":** You may have a springdoc version mismatch. Springdoc 3.x requires Spring Boot 4.x. This course uses Spring Boot 3.2.5 with springdoc 2.5.0.
- **Port 8080 already in use:** Free it with `lsof -ti:8080 | xargs kill -9` (macOS/Linux) or the equivalent Windows command.

Contract-First vs. Code-First

Reference for the two orders in which a spec and its implementation can be written, and the tradeoffs each one commits you to.

Learning Objectives

- *Distinguish contract-first (spec before code) from code-first (code before spec)*
- *Explain the tradeoff: contract-first buys parallelism at the cost of drift risk; code-first buys automatic sync at the cost of sequential work*
- *Identify when each approach is the right default*
- *Recognize that Postman makes the distinction concrete: mock server vs. live Swagger UI*

Core Idea

Contract-first means writing the OpenAPI specification before any implementation exists, then building code and client integrations against that agreed shape. Code-first means writing the implementation first and generating the specification from it afterward, which is what `springdoc-openapi` does on this course's Spring Boot API. Neither is universally correct; each trades coordination cost for drift risk in the opposite direction.

The Tradeoff

	Contract-first	Code-first
Sequence	Spec written first, code implements it	Code written first, spec generated from it
Source of truth	The `.yaml` file	The running code
Parallelism	Frontend, backend, and QA can all start immediately from the same contract	Frontend waits until code exists or an early stub is available
Drift risk	Spec and implementation can diverge unless enforced by contract testing	Docs auto-track the code, but the design decision happens implicitly while coding

Why 2026 Reporting Favors Contract-First

> ℹ️ **Note**
> Industry reporting on API-first adoption describes contract-first as the practice that persisted: teams write the OpenAPI spec first, and other teams start building against it, including mocking it in Postman, before a single line of server code exists. The payoff is parallelism. Separate teams, or in this course separate lab groups, build against the same agreed contract without waiting on each other. The cost is that nothing forces the eventual implementation to match the spec unless something checks it, which is why contract-first shops commonly add automated contract tests that compare live responses against the committed document.

Code-First Still Has a Place

> 💡 **Tip**
> Code-first remains a reasonable default for a small team or an early-stage API where hand-authoring a spec before any code exists is overhead without a payoff yet, since there is no second team waiting to build in parallel. The spec in that case is documentation of what already works rather than a plan for what should exist, and it never drifts because springdoc-openapi regenerates it from the code on every run.

Making the Distinction Concrete

Postman is where the difference stops being abstract. Point Postman at a mock server built purely from a Lab 1.2 spec, and you are consuming a contract-first API before any backend exists. Point Postman at the running Spring Boot service and its springdoc-openapi-generated Swagger UI, and you are consuming a code-first API where the docs are a byproduct of the implementation. Both are valid ways to expose the same information; the difference is which one existed first and which one is authoritative when they disagree.

Chapter Summary

Key Concepts	Key Takeaways
Contract-first	

Key Concepts	Key Takeaways
	Spec first, code second: buys parallelism, costs drift risk, needs contract tests
Code-first	Code first, spec generated: buys automatic sync, costs sequential work, good for small teams
Source of truth	`.yaml` file vs. running code. The difference is which one is authoritative when they disagree
Postman makes it concrete	Mock server from spec = contract-first; live Swagger UI = code-first

API Lifecycle: Overview & Versioning

Reference for how an API moves from a drafted contract to a retired endpoint, and how versioning lets that happen without breaking the consumers already depending on it.

Learning Objectives

- *Trace an API's lifecycle from design through deprecation and sunset*
- *Distinguish breaking from non-breaking changes and classify common modifications correctly*
- *Compare URI, header, and content-negotiation versioning strategies and their trade-offs*
- *Explain why deprecation is a three-phase process, not a single event*

Core Idea

An API is not a single artifact you ship once; it is a resource with a lifecycle. It gets designed against a contract, developed and tested, published as a live version, then maintained with a stream of changes until it is eventually deprecated and sunset. Versioning exists to manage that maintenance phase: it lets you evolve the API without pulling the ground out from under whoever already built against the version that's live.

> 💜 **Important**
> The distinction that matters most day to day is not "did I make a change," but "was that change breaking or non-breaking."

Breaking vs. Non-Breaking

	Non-breaking (safe within a version)	Breaking (requires a new version)
Fields	Adding a new optional response field	Removing or renaming an existing field
Endpoints	Adding a new endpoint	Changing a URL path or HTTP method
Validation	Relaxing a validation rule	Tightening a rule that rejects previously valid requests
Params		Making an optional parameter required

	Non-breaking (safe within a version)	Breaking (requires a new version)
	Adding a new optional request parameter	

A consumer's existing integration code should keep working through every non-breaking change. The moment it wouldn't, that change belongs in a new version, not a silent update to the old one.

Versioning Strategies

Spring Boot supports the same handful of strategies most REST APIs choose from.

Strategy	Mechanism	Strengths	Weaknesses
URI versioning	`/api/v1/orders` vs `/api/v2/orders` via separate `@RequestMapping` prefixes	Most visible; easy to read in logs, docs, and curl	URL churn on every major version
Header versioning	`X-API-Version: 2` custom header	Clean URLs	Harder to discover without reading docs
Content negotiation	`Accept: application/vnd.company.api.v2+json`	Most theoretically RESTful	Least discoverable in practice

> 💡 **Tip**
> In practice, public and partner-facing APIs lean on URI versioning because predictability matters more than purity when the consumers aren't in your control. Internal APIs, where the calling teams are known and can coordinate, can afford header or content-negotiation versioning instead.

A query parameter version is possible too, but it's the easiest for a client to simply forget to send.

Deprecation as a Process, Not an Event

Retiring an old version follows a three-phase pattern that shows up across most lifecycle-management guidance.

1. **Soft deprecation**: Keep the old version fully functional but add a warning: a `Deprecation` HTTP header, a changelog entry, or both.

2. **Hard deprecation**: Once most consumers have migrated, rate-limit or otherwise degrade the old version to push out stragglers.
3. **Final shutdown**: Remove it entirely, on a sunset date that was published in advance.

> 🛑 **Caution**
> Skipping the soft-deprecation phase, cutting a version off without warning, is one of the most commonly cited causes of broken partner integrations.

Common Pitfalls

> ⚠️ **Common Pitfalls**
> Bumping the major version for every change, including non-breaking ones, trains API consumers to stop paying attention to version numbers at all. A version strategy that stops at "we have a number in the URL" isn't really a strategy; it also needs to define how long old versions stay supported and how their deprecation gets communicated.

Chapter Summary

Key Concepts	Key Takeaways
API lifecycle	Design → develop → publish → maintain → deprecate → sunset. Versioning manages the maintenance phase
Breaking vs. non-breaking	Removing/renaming fields, changing paths/methods, tightening validation, and making params required are all breaking
Versioning strategies	URI (most visible), header (clean URLs), content negotiation (most RESTful). Choose per audience
Three-phase deprecation	Soft deprecation → hard deprecation → final shutdown on a published sunset date

Security for APIs: Standards

Reference for the three questions every request to a production API has to answer, and the standards that answer each one.

Learning Objectives

- *Separate API security into three distinct questions: authentication, authorization, and rate limiting*
- *Distinguish authentication (who are you) from authorization (what are you allowed to do)*
- *Describe the token-based authentication flow: short-lived JWT access tokens with refresh tokens*
- *Explain how rate limiting works with the token bucket algorithm and why a shared store matters in a cluster*
- *Connect security back to the OpenAPI contract from Day 2*

Core Idea

APIs now carry the large majority of web traffic, which also makes them the large majority of the attack surface. Security for an API breaks down into three separable questions, each answered by a different mechanism: who are you (authentication), what are you allowed to do (authorization), and how often are you allowed to do it (rate limiting).

> ⚠️ **Warning**
> Skipping any one of them doesn't make the other two redundant; a valid, authenticated caller can still do more damage than they should if authorization is missing, and an authorized caller can still overwhelm the system if nothing limits their request rate.

The OWASP API Security Top 10 is the industry-standard reference for the vulnerability classes that show up when one of these questions goes unanswered, and Broken Object Level Authorization, Broken Authentication, and Lack of Resources & Rate Limiting map directly onto the three questions above.

Authentication vs. Authorization

	Authentication	Authorization
Question answered	Who are you?	What are you allowed to do?
Common mechanism	OAuth 2.0 / OIDC, short-lived JWT access tokens	Role-based (RBAC), attribute-based (ABAC), or scope-based access control
Failure mode if skipped	Anyone can call the API as anyone	An authenticated caller acts outside their intended scope

Authentication in a modern API typically means a short-lived JWT access token, often around fifteen minutes, paired with a refresh token stored as an HTTP-only, Secure, `SameSite` cookie rather than in `localStorage`, which is straightforward to steal via a cross-site scripting bug.

Authorization then decides what that authenticated identity can actually touch.

> 💡 **Tip**
> Role-based access control, where permissions attach to a role like `ADMIN` or `USER`, is the simplest model to reason about and maps directly onto Spring Security's method-security annotations such as `@PreAuthorize("hasRole('ADMIN')")`. Attribute-based and scope-based models exist for cases where role alone isn't granular enough, at the cost of more implementation and testing effort.

Rate Limiting

Rate limiting defends against a caller, malicious or just misbehaving, making requests faster than the system can safely absorb. The token bucket algorithm is the common Java implementation, most often via the Bucket4j library: a bucket refills with tokens at a fixed rate, each request consumes one token, and an empty bucket means the request gets rejected. This naturally allows short bursts of activity while still enforcing a steady average rate over time.

Production systems often apply different limits per tier, tighter for unauthenticated endpoints, looser for authenticated ones, tighter again for sensitive mutation endpoints.

> 🛑 **Caution**
> A rate limiter that only lives in application memory has a well-known failure mode: behind a load balancer with several instances, each instance keeps its own bucket, silently multiplying the real limit by the number of instances. A shared store like Redis is what makes the limit real across a cluster, though a single-instance in-memory bucket is enough to learn the mechanism hands-on.

Where This Fits the Contract

The OpenAPI spec from Day 2 is where authentication requirements belong once they exist: security schemes are a first-class part of the OpenAPI document, not a side note. Adding basic security rules to the shared API is filling in a piece the Day 2 contract was always missing, not bolting on something unrelated to it.

Common Pitfalls

> ⚠️ **Common Pitfalls**
> Treating a valid token as sufficient on its own is the most common mistake; broken object- and function-level authorization, where an authenticated user reaches a resource or action they shouldn't, is one of the most cited real-world API vulnerability classes. Rate limiting only at a gateway and never per-user or per-endpoint fails to stop abuse from a caller who is otherwise fully authenticated.

Chapter Summary

Key Concepts	Key Takeaways
Three security questions	Authentication (who), authorization (what), rate limiting (how often). Skip one and the others don't save you
JWT access tokens	Short-lived (~15 min), paired with refresh tokens in HTTP-only Secure SameSite cookies, not localStorage
RBAC	Simplest authorization model; maps to `@PreAuthorize("hasRole('ADMIN')")` in Spring Security
Token bucket rate limiting	Burst-tolerant, average-enforcing; needs Redis across a cluster or the limit multiplies silently
Security in the contract	OpenAPI security schemes are first-class. Day 2's contract was always missing this piece

DevOps in API Development: Overview

Reference for how the OpenAPI contract from Days 1-2 becomes the artifact a real deployment pipeline gates on, and for the culture that makes automated delivery actually work.

Learning Objectives

- *Trace the OpenAPI spec through a four-stage CI/CD pipeline from trigger to promotion*
- *Distinguish Surefire (unit tests, `test` phase) from Failsafe (integration tests, `verify` phase)*
- *Define the four DORA metrics and what elite performance looks like*
- *Explain why "you build it, you run it" ownership matters more than the tooling*
- *Identify the four most common ways teams undermine their own pipeline investment*

Core Idea

API-first means the OpenAPI spec is designed and reviewed before implementation code exists, and every downstream activity, backend implementation, mock servers, generated docs, generated tests, reads from that same spec instead of a sequence of manual handoffs.

> ℹ️ **Note**
> Teams that work this way ship roughly 2.8 times faster and see 57 percent fewer integration defects than code-first teams. The spec you built in Days 1-2 for the quickstart project's `GreetingController` and `HealthController` endpoints is the contract a pipeline validates against on every push.

The Four-Stage Pipeline

A typical API CI/CD pipeline runs in four stages.

1. **Trigger**: Pulls the code, the spec, and test configuration together the moment someone pushes or opens a pull request.

2. **Build and spec validation**: Compiles the application and lints the OpenAPI document for missing schemas or undefined status codes, so a broken contract fails here rather than in production.
3. **Deploy to test environment and test**: Pushes the build to staging, confirms a health check passes, then runs unit tests, contract tests, and integration tests.
4. **Gate and promote**: Compares results against defined thresholds and either promotes the build or blocks it and notifies the team automatically.

> 💡 **Tip**
> Lab 2.3's JUnit5 suite, run from IntelliJ against the shared quickstart API, is a small-scale stand-in for that third stage: a real pipeline runs the same kind of test suite, just triggered by a commit instead of a click.

Surefire vs. Failsafe

Maven splits unit and integration tests across two plugins so a pipeline can gate on the fast tests before paying for the slow ones.

Plugin	Runs during	Test files	Purpose
Surefire	`test` phase	`*Test.java`	Fast, no external dependencies, run on every build
Failsafe	`verify` phase	`*IT.java`	Slower, environment-dependent, run only after unit tests pass

Lab 2.3's `GreetingControllerTest` and `HealthControllerTest` are exactly the kind of unit test Surefire picks up automatically by naming convention alone.

DORA Metrics and Deployment Health

Deployment frequency, lead time for changes, change failure rate, and mean time to restore are the four DORA metrics used to judge how healthy a delivery pipeline actually is.

Metric	Elite performance
Deployment frequency	Multiple times per day, sometimes per hour
Lead time for changes	Under one hour
Change failure rate	Zero to five percent

Metric	Elite performance
Mean time to restore	Under one hour

> ℹ️ **Note**
> Elite teams use canary releases and feature toggles to make frequent deployment safe rather than reckless.

You Build It, You Run It

The team that writes an API's code also deploys it, monitors it, and gets paged when it breaks. This ownership model, not the tooling, is what the DORA gains actually depend on; automating a pipeline on top of an unchanged handoff-based org structure rarely moves the numbers.

> 💜 **Important**
> Ownership only works with observability: structured logs, latency and error-rate metrics, and distributed traces, all sharing one correlation ID so an incident can be traced across all three signals instead of three disconnected dashboards.

Common Pitfalls

> ⚠️ **Common Pitfalls**
> Treating CI/CD as a tool suite bolted onto an unchanged team structure, instead of the culture and ownership shift it actually requires, is the most common way these gains fail to materialize. A close second is skipping OpenAPI spec validation as a pipeline gate and testing only the implementation, which lets the contract and the running API quietly drift apart. Running only unit tests in CI and treating integration or contract tests as optional defeats the entire point of splitting Surefire from Failsafe. And having Postman collections is not the same as having automated API testing; a collection only becomes a real gate once something runs it headlessly, via Newman, on every build.

Chapter Summary

Key Concepts	Key Takeaways
API-first pipeline	The OpenAPI spec gates every stage: build validation, test generation, and deployment promotion
	Trigger → build & validate spec → deploy & test → gate & promote

Key Concepts	Key Takeaways
Four-stage CI/CD	
Surefire vs. Failsafe	Surefire for fast unit tests (`*Test.java`), Failsafe for slower integration tests (`*IT.java`). Gate on the fast ones first
DORA metrics	Deployment frequency, lead time, change failure rate, MTTR. Elite teams deploy multiple times daily with <5% failure rate
You build it, you run it	Ownership drives the DORA gains; observability (logs, metrics, traces with one correlation ID) makes ownership possible

API Ecosystem

Reference for what surrounds a shipped API once the contract is real: codegen, layered testing, contract testing, and where security plugs into the pipeline.

Learning Objectives

- *Explain how the OpenAPI spec drives codegen, mock servers, and contract tests, not just documentation*
- *Describe the three-layer test strategy and where each layer runs in a real pipeline*
- *Distinguish contract testing from unit and integration testing, and compare Spring Cloud Contract with Pact*
- *Map security controls to their correct pipeline stages*
- *Articulate the case for deep API knowledge: why the costliest outages trace back to an ambiguous contract, not a buggy line*

Core Idea

82 percent of organizations now run some level of API-first practice, and 25 percent operate as fully API-first; this is already how the industry works, not an aspirational ideal. The OpenAPI spec you wrote against the quickstart project in Days 1-2 is the same artifact that drives everything in this session: codegen, mock servers for consumers, and the contract tests that check your implementation against what it promises.

> 💜 **Important**
> Losing sight of that connection, treating the spec as documentation rather than a live input other tools consume, is how teams give up most of the API-first benefit.

API-First in Practice: Codegen

Once a contract exists, tools like OpenAPI Generator read it directly to produce server stubs and client SDKs, so design and implementation stay in sync by construction rather than by someone remembering to update both. Frontend teams can build against mocked responses generated from

the same spec while backend teams implement the real logic, a decoupling organizations report cutting development time by up to half.

> ℹ️ **Note**
>
> The spec springdoc-openapi generates from `GreetingController` and `HealthController` in this course is exactly the kind of document a generator would consume to produce a client library, without hand-writing either side twice.

The Layered Test Strategy

Pipelines in 2026 typically run tests in three layers rather than one flat suite: unit and contract tests on every push, integration, security, and performance tests inside the CI/CD pipeline itself, and synthetic monitors continuously against production. Lab 2.3's JUnit5 suite occupies the first layer, running locally in IntelliJ exactly as it would run automatically on every commit in a real pipeline.

Layer	When it runs	What it checks
Unit and contract	Every push, pre-commit	Does this code and this contract still agree with themselves
Integration, security, performance	Inside CI/CD	Does the assembled system behave correctly under real conditions
Synthetic monitoring	Continuously, in production	Is the live API still behaving as documented, right now

Contract Testing

Unit and Postman tests both check a service against its own expectations, not against what its consumers actually depend on. Contract testing closes that gap.

Tool	Model	Best fit
Spring Cloud Contract	Provider-owned contracts in Groovy or YAML; generates producer verification tests and a runnable stub	All-Spring-Boot shops reusing existing Maven or Gradle infrastructure, no broker required
Pact	Consumer-driven; the consumer defines the expected interaction, the provider verifies it via a Pact Broker	Polyglot environments, or where a central "can-i-deploy" gate matters

> 💡 **Tip**
> Spring Cloud Contract is the more natural fit for this course's all-JVM stack, but Pact is worth recognizing by name since many real organizations mix services built on different platforms.

Where Security Sits in the Pipeline

Day 3's concept docs already cover authentication, authorization, and rate limiting in depth; this session only needs to show where those controls plug into delivery.

- **Pre-commit hooks** catch secrets before code is even pushed
- **CI gates** run dependency and static analysis scans on pull requests
- **CD gates** scan container images before they reach a registry
- **Scheduled dynamic scans** run against staging or production on a schedule rather than blocking every commit

> ⚠️ **Warning**
> Automated scanners still cannot reason about business logic on their own, so a flaw like broken object-level authorization needs a targeted test case; a clean scanner report alone won't catch it.

The Case for Deep API Knowledge

The costliest API outages rarely trace back to one buggy line; they trace back to an ambiguous or unenforced contract where one overlooked assumption compounds once traffic scales. An engineer who only writes endpoints inherits that risk; one who reasons about the full lifecycle, design, versioning, documentation, security, and testing together, catches it before it ships.

The capstone in Lab 2.4 is a compressed rehearsal of exactly that: it asks you to keep the spec, the security rules, and the tests all in sync on the same shared API, which is the entire argument of this session in miniature.

Common Pitfalls

> ⚠️ **Common Pitfalls**
>
> Letting the spec drift from the implementation because it is treated as documentation rather than a live contract undoes most of what API-first buys you. Running Newman only by hand and never wiring it into CI means regressions are caught by consumers instead of the pipeline. Gating every single commit on slow dynamic scans, instead of reserving them for scheduled runs, either stalls delivery or gets bypassed under deadline pressure.

Chapter Summary

Key Concepts	Key Takeaways
The spec is a live artifact	OpenAPI drives codegen, mock servers, and contract tests, not just docs
Three-layer test strategy	Unit/contract on every push, integration/security/performance in CI/CD, synthetic monitoring in production
Contract testing	Closes the gap between "the service works" and "it works for its consumers." Spring Cloud Contract for JVM, Pact for polyglot
Security in the pipeline	Secrets at pre-commit, static analysis at CI, image scans at CD, dynamic scans on a schedule
Deep API knowledge	The costliest outages trace to ambiguous contracts, not buggy lines. Lab 2.4 is the capstone rehearsal

Installation Instructions

Installation instructions for the **Fundamentals of API Development** course.

Click on the link for your operating system to view the detailed setup guide:

Platform	Installation Guide
macOS	macOS Install Guide
Windows	Windows Install Guide

What You Will Install

Category	Tools / Packages
JDK	OpenJDK 17+ (recommended: JDK 21)
Build Tool	Apache Maven 3.9+
IDE	IntelliJ IDEA (Community or Ultimate) **or** VS Code with Java & Spring Boot Extension Packs
API Client	Postman
Version Control	Git

> *The course labs are built against a Spring Boot REST API, tested with JUnit5 and Postman. You can use IntelliJ IDEA or VS Code as your IDE; both are fully supported. See the Course Outline for the full course outline.*

Verify Your Installation

After completing the OS-specific install guide, use the quickstart-project to confirm every tool works end to end:

1. Open the quickstart-project in IntelliJ IDEA or VS Code as a Maven project.
2. Build and run it with Maven (`mvn spring-boot:run`).
3. Call its REST endpoints from Postman using the provided collection.
4. Run the JUnit5 tests (`mvn test`) from your IDE or the command line.

Full steps are in the quickstart-project README.

If port 8080 is already in use (e.g. a previous `mvn spring-boot:run` was left running), see Free Up Port 8080 for macOS/Linux and Windows commands to kill it.

Quick Verification Checklist

- [ ] `java -version` shows 17+ (21 recommended)
- [ ] `mvn -version` works and reports the same Java version
- [ ] `git --version` works
- [ ] IntelliJ IDEA opens and recognizes `quickstart-project` as a Maven project
- [ ] VS Code (with Java extensions) opens and recognizes `quickstart-project` as a Maven project
- [ ] Postman is installed and can send a request
- [ ] `mvn spring-boot:run` starts `quickstart-project` on http://localhost:8080
- [ ] `GET http://localhost:8080/api/v1/health` returns `{"status":"UP", ...}` (via browser, curl, or Postman)
- [ ] The Postman collection's requests all pass their tests
- [ ] `mvn test` passes all JUnit5 tests

Installation Guide (macOS)

Development environment setup on macOS for the **Fundamentals of API Development** course: JDK, Maven, IntelliJ IDEA (or VS Code), Postman, and Git.

1. Install Homebrew (if not already installed)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/
install/HEAD/install.sh)"
```

Verify:

```
brew --version
```

2. Install Java Development Kit (JDK)

Install OpenJDK 17 or higher (recommended: JDK 21):

```
brew install openjdk@21
```

Create a symbolic link for system-wide access:

```
sudo ln -sfn /usr/local/opt/openjdk@21/libexec/openjdk.jdk /Library/
Java/JavaVirtualMachines/openjdk-21.jdk
```

Add OpenJDK to your PATH and set `JAVA_HOME` in your shell profile:

```
echo 'export PATH="/usr/local/opt/openjdk@21/bin:$PATH"' >> ~/.zshrc
echo 'export JAVA_HOME=`/usr/libexec/java_home -v 21`' >> ~/.zshrc
source ~/.zshrc
```

On Apple Silicon (M1/M2/M3), Homebrew usually installs under `/opt/homebrew` *instead of* `/usr/local`*. If* `brew --prefix` *prints* `/opt/homebrew`*, adjust the paths above:*

`bash echo 'export PATH="$(brew --prefix)/opt/openjdk@21/bin:$PATH"' >> ~/.zshrc sudo ln -sfn "$(brew --prefix)/opt/openjdk@21/libexec/openjdk.jdk" /Library/Java/JavaVirtualMachines/openjdk-21.jdk source ~/.zshrc`

Validate:

```
java -version
javac -version
```

3. Install Maven

```
brew install maven
```

Verify:

```
mvn -version
```

You should see Maven version information along with the Java details from Step 2.

4. Install Git

```
brew install git
```

Configure Git with your information:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Verify:

```
git --version
```

5. Install IntelliJ IDEA

Download IntelliJ IDEA (Community or Ultimate) from https://www.jetbrains.com/idea/download/, or install via Homebrew:

```
brew install --cask intellij-idea-ce
```

Open IntelliJ IDEA once after installing so it finishes its first-run setup.

6. Install VS Code (alternative IDE)

If you prefer VS Code over IntelliJ IDEA, install it and the required Java extensions. Skip this section if you installed IntelliJ in step 5; you only need one IDE.

Download VS Code from https://code.visualstudio.com/, or install via Homebrew:

```
brew install --cask visual-studio-code
```

Open VS Code once after installing so it finishes its first-run setup.

Install Java Extensions

Open VS Code, press `Cmd+Shift+X` to open the Extensions view, and install these three extensions:

Extension	Marketplace ID	Purpose
Extension Pack for Java	`vscjava.vscode-java-pack`	Bundles Language Support for Java, Debugger, Test Runner, Maven, and Project Manager: everything needed to work with the course's Spring Boot projects
Spring Boot Extension Pack	`vmware.vscode-boot-dev-pack`	Spring Tools for VS Code: content assist for `application.properties` / `.yml`, Spring-specific code navigation (`@/` for request mappings, `@+` for beans), and a Spring Boot Dashboard to start, stop, and debug projects
Code Runner	`formulahendry.vscode-code-runner`	Run Java and many other languages directly from the editor with a single click or keyboard shortcut

Alternatively, install all three from the terminal:

```
code --install-extension vscjava.vscode-java-pack
code --install-extension vmware.vscode-boot-dev-pack
code --install-extension formulahendry.vscode-code-runner
```

To open the quickstart project in VS Code:

```
code ../quickstart-project
```

VS Code will detect the `pom.xml` and prompt you to import the Maven project. Accept it, and the Java extension pack will configure the classpath automatically.

7. Install Postman

Download Postman from https://www.postman.com/downloads/, or install via Homebrew:

```
brew install --cask postman
```

Open Postman once and sign in (or continue without an account) so it's ready for the labs.

8. Verification

Run this end-to-end check before class:

```
java -version && echo "✓ Java OK"
mvn -version && echo "✓ Maven OK"
git --version && echo "✓ Git OK"
code --version && echo "✓ VS Code OK"
```

Then use the quickstart-project to confirm IntelliJ, Maven, and Postman work together end to end. See the quickstart-project README for the steps, or the checklist in the Installation Instructions.

9. Troubleshooting

- `java` / `mvn` **not found after install**: Restart your terminal, or re-run `source ~/.zshrc`.
- **Wrong Java version picked up**: Run `/usr/libexec/java_home -V` to list installed JDKs, and confirm `JAVA_HOME` points at the one you want.
- **IntelliJ doesn't detect the JDK**: In IntelliJ, go to **File > Project Structure > SDKs** and add the JDK path from `echo $JAVA_HOME`.

- **Maven can't download dependencies**: Check your internet connection, or ask your DevOps team for an organization-specific `~/.m2/settings.xml` if you're behind a corporate proxy.

Installation Guide (Windows)

Development environment setup on Windows 10/11 for the **Fundamentals of API Development** course: JDK, Maven, IntelliJ IDEA (or VS Code), Postman, and Git.

1. Install Java Development Kit (JDK)

Download and install Java 17 or higher (recommended: JDK 21).

1. **Download JDK**:
2. Visit Oracle Java Downloads, OpenJDK, or Adoptium
3. Download the Windows x64 installer (`.msi`)
4. **Install JDK**:
5. Run the installer and follow the wizard
6. Note the installation path (typically `C:\Program Files\Java\jdk-21`)

Set JAVA_HOME Environment Variable

1. Press `Windows + R` , type `sysdm.cpl` , press Enter, then open the **Advanced** tab and click **Environment Variables**.
2. Under "System Variables", click **New**:
3. Variable name: `JAVA_HOME`
4. Variable value: `C:\Program Files\Java\jdk-21` (adjust if different)
5. Find **Path** in System Variables, click **Edit**, then **New**, and add: `%JAVA_HOME%\bin`

6. Click **OK** to close all dialogs.

Verify in a new Command Prompt:

```
java -version
javac -version
```

2. Install Maven

1. **Download**: Visit Apache Maven Downloads and download the Binary zip archive (e.g. `apache-maven-3.9.6-bin.zip`).
2. **Extract**: Extract to `C:\Program Files\Maven`, so the final path is `C:\Program Files\Maven\apache-maven-3.9.6`.

Set Maven Environment Variables

1. Open Environment Variables (same as Step 1).
2. Under "System Variables", click **New**:
3. Variable name: `M2_HOME`
4. Variable value: `C:\Program Files\Maven\apache-maven-3.9.6`
5. Find **Path**, click **Edit**, then **New**, and add: `%M2_HOME%\bin`
6. Click **OK** to close all dialogs.

Verify in a new Command Prompt:

```
mvn -version
```

You should see Maven version information along with the Java details from Step 1.

3. Install Git

1. Download from Git for Windows and run the installer (default settings are fine).
2. Ensure "Git from the command line and also from 3rd-party software" is selected during install.
3. Configure your identity:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Verify:

```
git --version
```

4. Install IntelliJ IDEA

Download IntelliJ IDEA (Community or Ultimate) from https://www.jetbrains.com/idea/download/ and run the installer. Open IntelliJ IDEA once after installing so it finishes its first-run setup.

5. Install VS Code (alternative IDE)

If you prefer VS Code over IntelliJ IDEA, install it and the required Java extensions. Skip this section if you installed IntelliJ in step 4; you only need one IDE.

Download VS Code from https://code.visualstudio.com/ and run the installer (default settings are fine).

Open VS Code once after installing so it finishes its first-run setup.

Install Java Extensions

Open VS Code, press `Ctrl+Shift+X` to open the Extensions view, and install these three extensions:

Extension	Marketplace ID	Purpose
Extension Pack for Java	`vscjava.vscode-java-pack`	Bundles Language Support for Java, Debugger, Test Runner, Maven, and Project Manager: everything needed to work with the course's Spring Boot projects
Spring Boot Extension Pack	`vmware.vscode-boot-dev-pack`	Spring Tools for VS Code: content assist for `application.properties` / `.yml` , Spring-specific code navigation (`@/` for request mappings, `@+` for beans), and a Spring Boot Dashboard to start, stop, and debug projects
Code Runner	`formulahendry.vscode-code-runner`	Run Java and many other languages directly from the editor with a single click or keyboard shortcut

Alternatively, install all three from a terminal (Command Prompt or PowerShell):

```
code --install-extension vscjava.vscode-java-pack
code --install-extension vmware.vscode-boot-dev-pack
code --install-extension formulahendry.vscode-code-runner
```

To open the quickstart project in VS Code:

```
code ..\quickstart-project
```

VS Code will detect the `pom.xml` and prompt you to import the Maven project. Accept it, and the Java extension pack will configure the classpath automatically.

6. Install Postman

Download Postman from https://www.postman.com/downloads/ and run the installer. Open Postman once and sign in (or continue without an account) so it's ready for the labs.

7. Verification

Run this end-to-end check in a new Command Prompt:

```
java -version && echo Java OK
mvn -version && echo Maven OK
git --version && echo Git OK
code --version && echo VS Code OK
```

Then use the quickstart-project to confirm IntelliJ, Maven, and Postman work together end to end. See the quickstart-project README for the steps, or the checklist in the Installation Instructions.

8. Troubleshooting

- `java` / `mvn` **not recognized**: Restart Command Prompt after setting environment variables, and double-check `Path` includes `%JAVA_HOME%\bin` and `%M2_HOME%\bin`.
- **Wrong Java version picked up**: Run `echo %JAVA_HOME%` and confirm it points at the JDK you intend to use.
- **IntelliJ doesn't detect the JDK**: In IntelliJ, go to **File > Project Structure > SDKs** and add the JDK path from `echo %JAVA_HOME%`.
- **Maven can't download dependencies**: Check your internet connection, or ask your DevOps team for an organization-specific

`settings.xml` (place it in `C:\Users\%USERNAME%\.m2\settings.xml`) if you're behind a corporate proxy.

- **Path issues**: Avoid spaces in folder names where possible, and use backslashes (`\`) for Windows paths.

REST API Quick Reference: HTTP Methods

> ***KEY CONCEPT*** *Safe means the call does not change server state. Idempotent means repeating the same call leaves the server in the same state. The second, third, and hundredth call have the same effect as the first.*

Method	Safe	Idempotent	Use	Success Code
GET	yes	yes	Read a resource or collection	`200 OK`
HEAD	yes	yes	Read headers only (no body)	`200 OK`
POST	no	no	Create a subordinate resource, or a non-CRUD action	`201 Created` (with `Location` header)
PUT	no	yes	Replace the entire resource at a known URI	`200 OK` or `204 No Content`
PATCH	no	no	Partial update	`200 OK` or `204 No Content`
DELETE	no	yes	Remove a resource	`204 No Content`

Choosing the Right Method

If you want to...	Use	Example
Fetch a list of orders	`GET`	`GET /api/v1/orders`
Fetch one order	`GET`	`GET /api/v1/orders/42`
Create a new order	`POST`	`POST /api/v1/orders`
Replace order 42 entirely	`PUT`	`PUT /api/v1/orders/42`
Update only the status of order 42	`PATCH`	`PATCH /api/v1/orders/42`
Remove order 42	`DELETE`	`DELETE /api/v1/orders/42`
Check if order 42 exists (no body needed)	`HEAD`	`HEAD /api/v1/orders/42`

Why the Guarantees Matter

Generic infrastructure (caches, proxies, load balancers, CDNs) makes decisions based on these guarantees.

Guarantee	What infrastructure does with it
`GET` is safe	A cache can serve a `GET` response without asking the server
`PUT` is idempotent	A proxy can retry a failed `PUT` without asking the client

Guarantee	What infrastructure does with it
`DELETE` is idempotent	A retry on `DELETE` should not return `500`. The resource is already gone, and that is fine
`POST` is neither	A proxy must not retry a failed `POST` without the client's knowledge

***KEY CONCEPT** A `GET` that mutates server state and a `DELETE` that returns `500` on the second call are the two violations that produce the most mysterious production behaviour, because retries and caches are built on the assumption that neither happens.*

REST API Quick Reference: HTTP Status Codes

> ***KEY CONCEPT*** *The status code is the part of your response that machines act on. Get it right and generic clients, caches, gateways, and dashboards work for free. Answer* `200` *for everything and every consumer has to write custom handling for you.*

Status Code Families

Family	Meaning	What the caller should do	Codes to Know
2xx	It worked	Process the response	`200 OK`, `201 Created`, `202 Accepted`, `204 No Content`
3xx	Look elsewhere	Follow the redirect, or use your cache	`301 Moved Permanently`, `304 Not Modified`
4xx	Your request was wrong	Fix the request; do not retry unchanged	`400`, `401`, `403`, `404`, `405`, `409`, `422`, `429`
5xx	Something failed on my side	Retry with backoff	`500`, `502`, `503`, `504`

Common Status Codes

Code	Name	When to Use
`200`	OK	Standard success for `GET`, `PUT`, `PATCH`
`201`	Created	Resource created by `POST`; always include a `Location` header
`202`	Accepted	Async operation started; return a status URL the caller can poll
`204`	No Content	Success with no response body (`DELETE`, or `PUT` / `PATCH` returning nothing)
`301`	Moved Permanently	The resource has a new permanent URI
`304`	Not Modified	The caller's cached copy is still fresh (used with `ETag` / `If-None-Match`)
`400`	Bad Request	The server could not parse the request body or query
`401`	Unauthorized	No valid credentials; the server does not know who is calling
`403`	Forbidden	Credentials are valid but the caller lacks permission
`404`	Not Found	The resource does not exist

Code	Name	When to Use
`405`	Method Not Allowed	The HTTP method is not supported for this endpoint; send an `Allow` header
`409`	Conflict	The request conflicts with the current state (e.g. duplicate, version mismatch)
`422`	Unprocessable Content	The request parsed correctly but failed semantic validation
`429`	Too Many Requests	Rate limit exceeded; send a `Retry-After` header
`500`	Internal Server Error	An unexpected server-side failure
`502`	Bad Gateway	An upstream server returned an invalid response
`503`	Service Unavailable	The server is temporarily down (maintenance, overload)

The Three Pairs People Get Wrong

Pair	Distinction
`401` **vs** `403`	`401` = I do not know who you are (unauthenticated). `403` = I know who you are, and you may not do that (forbidden).
`400` **vs** `422`	`400` = I could not parse your request (malformed JSON, missing field). `422` = I parsed it, but it fails business rules (negative amount, invalid state transition).
`200` **vs** `4xx`	Returning `200 OK` with `{"success": false}` defeats every retry policy, every gateway metric, every cache, and every generic client at once. It is the single most damaging shortcut.

RFC 9457: Standard Error Shape

Serve errors as `application/problem+json`:

```
{
  "type": "https://api.example.com/problems/insufficient-funds",
  "title": "Insufficient funds",
  "status": 409,
  "detail": "Account 42 has a balance of 12.50 and the transfer
requires 100.00",
  "instance": "/api/v1/accounts/42/transfers/8871"
}
```

Spring supports this via `ProblemDetail` (Framework 6.0+).

REST API Quick Reference: Resource Naming Rules

> ***KEY CONCEPT*** *These rules are not aesthetics. Each one exists so a caller can guess your next endpoint correctly without reading your documentation. Listed in order of how many defects each one catches.*

The Eight Rules

#	Rule	Correct	Incorrect	What It Prevents
1	**Nouns, not verbs**	`/orders`	`/getOrders`, `/createOrder`	The HTTP method is already the verb; duplicating it in the path means every consumer has to learn your vocabulary
2	**Plural collections**	`/accounts/42`	`/account/42`	Inconsistency: one rule applied everywhere beats a mixture of singular and plural
3	**Hierarchy means containment**	`/customers/17/orders`	`/orders?customerid=17`	A flat query-string API forces every consumer to read docs to find related resources
4	**lowercase-with-hyphens**	`/purchase-orders`	`/purchaseOrders`, `/purchase_orders`	camelCase and snake_case are harder to read in URLs and break convention with HTTP headers
5		`/orders?status=shipped&page=2`	`/orders/shipped`	Statuses and filters as path

#	Rule	Correct	Incorrect	What It Prevents
	Path identifies, query filters			segments create an explosion of endpoints for every combination
6	**No extensions, no trailing slash**	`/orders/42`	`/orders/42.json` , `/orders/`	Content negotiation belongs in the `Accept` header; trailing slashes create duplicate resources
7	**Version explicitly**	`/api/v1/orders`	`/orders`	Without a version segment, every change is potentially breaking. You cannot evolve the API safely
8	**Do not leak the schema**	`/customers/17`	`/tbl_cust_master/17`	Publishing your database schema guarantees that a refactor becomes a breaking change to the public contract

Examples: Before and After

Before	After	Rules Violated
`GET /getOrderById?id=42`	`GET /api/v1/orders/42`	1, 5, 7
`POST /createOrder`	`POST /api/v1/orders`	1, 7
`GET /order/42`	`GET /api/v1/orders/42`	2, 7
`GET /purchaseOrders`	`GET /api/v1/purchase-orders`	4, 7
`GET /orders/42.json`	`GET /api/v1/orders/42`	6, 7
`GET /orders/`	`GET /api/v1/orders`	6, 7

Before	After	Rules Violated
`GET /tbl_cust_master/17`	`GET /api/v1/customers/17`	8, 7
`GET /deleteOrder?id=42`	`DELETE /api/v1/orders/42`	1, 5, 7

The One Defensible Exception

Actions that do not map cleanly to CRUD, like cancelling an order, are genuinely awkward in strict REST.

Approach	Example
Model the action as a resource	`POST /api/v1/orders/42/cancellation`
Accept a verb and document it	`POST /api/v1/orders/42/cancel`

Either is defensible. Silence, shipping it with no documentation, is not.

REST API Quick Reference: Common Anti-Patterns

> ***KEY CONCEPT*** *These are the patterns that cause the most production incidents, debugging hours, and consumer frustration. Each one breaks a guarantee that generic infrastructure depends on.*

1. Burying Errors Inside `200 OK`

The mistake: Returning `200 OK` with a body like `{"success": false, "error": "Insufficient funds"}`.

What breaks:

System	What goes wrong
Retry policies	The client never retries. It got a `200`, so it assumes success
Caches	The error response gets cached and served to other callers
Gateways and load balancers	Error-rate dashboards report 0% failures while a third of calls actually fail
Monitoring and alerting	On-call engineers are never paged for failing requests

The fix: Use the correct status code. A business rule violation is `409 Conflict` or `422 Unprocessable Content`, not `200 OK`.

2. `GET` Requests That Mutate State

The mistake: `GET /api/v1/orders/42/approve` changes server state.

What breaks:

System	What goes wrong
Caches and CDNs	The mutation response gets cached; subsequent `GET`s serve the stale (pre-mutation) state
Web crawlers and prefetch	A crawler following links accidentally approves every order it encounters
Retry logic	A proxy retrying a failed `GET` approves the order twice

The fix: Mutations belong on `POST`, `PUT`, `PATCH`, or `DELETE`, never on `GET`.

3. Non-Idempotent `DELETE`

The mistake: `DELETE /api/v1/orders/42` returns `500` on the second call because the resource is already gone.

What breaks:

System	What goes wrong
Retry logic	A proxy retrying a failed `DELETE` gets a `500` and escalates. The first call succeeded, but the caller never knows
Client error handling	The client cannot distinguish "the delete failed" from "the resource was already deleted"

The fix: `DELETE` is defined as idempotent. The second call to `DELETE /api/v1/orders/42` should return `204 No Content` or `404 Not Found`, never `500`.

4. Leaking Internal Fields to the Wrong Audience

The mistake: Serialising an internal database record directly to a public endpoint.

Leaked field	Why it matters
`taxId`, `ssn`	Compliance violation (PII exposure)
`internalRiskTier`	Reveals fraud-detection thresholds to attackers
`assignedAnalystEmail`	Enables social engineering against internal staff
`storageShardKey`	Publishes your infrastructure topology. A re-shard becomes a breaking change to the public API

The fix: Project your resources per audience. An Open endpoint returns 3 fields; a Partner endpoint returns 5; an Internal endpoint returns all 9. The same resource, three deliberate projections.

5. Skipping Versioning

The mistake: Evolving an API without versioning: renaming a field, changing a type, or removing a parameter on the live endpoint.

What breaks:

Change	Consumer impact
Rename `owner` to `accountHolder`	Every consumer's deserialisation breaks at compile time or runtime

Change	Consumer impact
Change `status` from `String` to `int`	Type mismatch. Consumer code that expected `"ACTIVE"` now gets `1`
Make an optional query parameter required	Existing callers that omit it start getting `400`

The fix: Version your API explicitly. `/api/v1/orders` is the pragmatic default. The question is never "did something change." It is "does the existing consumer's code still work unmodified."

6. Ignoring the `Allow` Header on `405`

The mistake: Returning `405 Method Not Allowed` without an `Allow` header.

The fix: Always include `Allow: GET, HEAD, OPTIONS` (or whichever methods are supported) so the caller knows what to try next without reading documentation.

Quick Decision Table

You want to...	Do this	Not this
Signal a business rule violation	Return `409` or `422`	Return `200` with `{"success": false}`
Let a client poll for status	Return `202 Accepted` with a status URL	Return `200` and make them guess
Tell a caller to slow down	Return `429` with `Retry-After`	Return `200` and hope they notice
Remove a resource	Return `204` on success, `404` if already gone	Return `500` on the second call
Expose data to the public	Project only the safe fields	Serialise the internal record directly

About the Author

Kangeyan Passoubady (Kangs)

Kangeyan Passoubady is a Principal AI Architect, engineering leader, and the founder of Kavin School, a training organization focused on software development, testing, and artificial intelligence. With over 27 years of expertise in the software industry, he specializes in designing modular, high-signal automation patterns for distributed, cloud-native systems, and has spearheaded engineering transformations at organizations including Delta Dental Insurance, ServiceNow, Socotra, and Invitae.

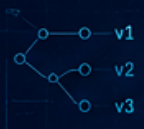
An expert in building AI-native agentic workflows, Kangeyan integrates Large Language Models into daily engineering processes to enhance developer throughput and code quality. He possesses deep expertise in API testing, microservices validation, and observability, ensuring that complex software services run reliably at scale. A passionate advocate for open-source and modern engineering practices, he is the creator of several agentic tooling repositories designed to bridge AI with practical development workflows.

Through Kavin School and his extensive corporate leadership, Kangeyan has mentored and trained thousands of developers across the globe, guiding them to adopt shift-left quality mindsets, build resilient automated architectures, and master the fundamentals of modern API development.

- **Website**: kavinschool.com
- **LinkedIn**: linkedin.com/in/kpassoubady
- **GitHub**: github.com/kpassoubady

Index

`



ABOUT THE BOOK

Fundamentals of API Development is a comprehensive guide to designing, building, securing, and managing modern APIs. It covers the core architectural principles, industry standards, and practical techniques required to create robust, scalable, and secure APIs.

From REST, SOAP, and RPC to OpenAPI/Swagger, authentication, and microservices, this book provides end-to-end coverage of the API lifecycle.

- API Architecture and Design Principles
- REST, SOAP, RPC, and Modern API Styles
- OpenAPI Specification and Documentation
- Authentication, Authorization, and Security Best Practices
- Microservices and Service-to-Service Communication
- Testing Strategies for APIs
- CI/CD, DevOps, and API Deployment
- Monitoring, Observability, and Performance
- API Versioning, Governance, and Lifecycle Management

ABOUT THE AUTHOR

Kangeyan Passoubady is a technology architect, engineer, instructor, and trainer with over 25 years of experience in software engineering, test automation, and API technologies. He has led engineering teams, designed enterprise-scale automation frameworks, and architected API solutions across multiple domains.

He is passionate about building quality engineering cultures and sharing practical knowledge through training and mentoring.

Kangs | Kavin School

Empowering Engineers. Building the Future.

ISBN 978-1-963452-01-8
90000
9 781963 452018